

PERSONAL EXPENSES TRACKER

Project Based Learning (PBL) Report

for the course

Web Technologies-20CS22004

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

By

P VARUN 24R11A05W1

P LIRITH 24R11A05W0

K NAVEEN 24R11A05U9

Under the guidance of

Mr P.CHANDRA SEKHAR REDDY



Department of Computer Science and Engineering

Accredited by NBA

Geethanjali College of Engineering and Technology

(UGC Autonomous)

(Affiliated to J.N.T.U.H, Approved by AICTE, New Delhi)

Cheeryal (V), Keesara (M), Medchal.Dist.-501 301.

A.Y 2025-26 (Sem-II)

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to our institution, Geethanjali College of Engineering and Technology, for providing me with the opportunity to carry out this Project Based Learning (PBL) work on the Personal Expenses Tracker.

I would like to express my heartfelt thanks to my project guide P. Chandra Sekhar Reddy for his continuous guidance, valuable suggestions, and constant encouragement throughout the development of this project. His support and mentorship played a crucial role in the successful completion of this work.

I would also like to thank the faculty members of the Department of Computer Science and Engineering for their support and for providing the necessary resources to complete this project effectively.

Finally, I extend my gratitude to my friends and peers who contributed directly or indirectly to this project through their suggestions and support.

TABLE OF CONTENTS

S.No.	Contents	Page No
1	Abstract	4
2	Introduction	5
3	System Design	8
4	Implementation	16
5	Conclusion	27
6	References	28

1.ABSTRACT

The Personal Expenses Tracker is a dynamic web-based application developed using HTML, CSS, and JavaScript to help users efficiently manage their daily financial transactions. The system provides an interactive interface where users can add income and expense details, categorize transactions, and track their financial status in real time. It incorporates browser-based storage using localStorage to ensure data persistence without requiring a backend database.

The application features a visually appealing user interface with modern design elements such as gradient backgrounds, glassmorphism effects, and dark mode support. It also integrates data visualization using the Chart.js library to present category-wise expense distribution through a pie chart. Users can filter transactions, view summaries of total income, expenses, and balance, and manage records efficiently through add and delete operations.

Overall, the system enhances financial awareness by providing real-time insights and an intuitive user experience, demonstrating the effective use of front-end web technologies in building functional and responsive applications.

2.INTRODUCTION

2.1 About the project

The Personal Expenses Tracker is a web-based application developed to simplify the process of managing daily financial transactions. The project focuses on providing an easy-to-use and interactive platform where users can record their income and expenses, categorize them, and monitor their financial status in real time.

This application is built using core web technologies such as HTML, CSS, and JavaScript, making it lightweight and accessible through any modern web browser. It utilizes browser storage (localStorage) to save user data, eliminating the need for a backend database while still ensuring data persistence across sessions.

The system includes features such as transaction entry, category-based classification, real-time balance calculation, and dynamic data visualization. The integration of charts using the Chart.js library helps users understand their spending patterns through graphical representation. Additionally, the application provides filtering options to view specific categories and allows users to delete transactions easily.

The user interface is designed with modern styling techniques, including gradient backgrounds, glassmorphism effects, and a dark mode option, enhancing both aesthetics and usability. The application also incorporates responsive design principles, ensuring compatibility across different devices.

Overall, this project demonstrates how front-end technologies can be effectively used to develop a fully functional and visually appealing financial management tool, helping users gain better control over their personal finances

2.2 Objectives of the Project

- To develop a web-based application for managing personal financial transactions efficiently
- To enable users to record both income and expenses in a structured manner
- To categorize transactions (Food, Travel, Bills, Others) for better financial analysis
- To provide real-time calculation of total income, total expenses, and remaining balance
- To implement data persistence using browser localStorage without requiring a backend
- To visualize spending patterns using charts with the help of Chart.js
- To allow users to filter transactions based on categories
- To provide functionality to add and delete transactions dynamically
- To design a user-friendly and interactive interface with modern UI elements
- To include additional features like dark mode and responsive design for better user experience

2.3 Project Outcomes

- The system successfully enables users to record and manage income and expenses in a structured and efficient manner
- Users can view real-time financial summaries, including total income, total expenses, and current balance
- The application provides category-wise tracking, helping users understand where their money is being spent
- The integration of Chart.js allows users to visualize spending patterns through interactive pie charts

- The use of `localStorage` ensures data persistence, allowing users to retain their data even after refreshing the page
- The system supports dynamic operations such as adding and deleting transactions with immediate updates
- The project demonstrates effective use of frontend technologies (HTML, CSS, JavaScript) to build a complete application
- The modern UI design, including dark mode and animations, enhances the overall user experience
- Overall, the project helps users develop better financial awareness and budgeting habits

2.4 Scope of the Project

The Personal Expenses Tracker is designed to provide a simple and efficient solution for managing daily financial activities using web technologies. The scope of this project primarily focuses on front-end development and client-side data handling.

The application allows users to record income and expenses, categorize transactions, and view real-time summaries of their financial status. It includes features such as transaction management, category-based filtering, and graphical representation of data using the Chart.js library. The use of browser `localStorage` ensures that user data is stored and retrieved efficiently without requiring server-side support.

The project is limited to a single-user environment and operates entirely within the browser. It does not include advanced features such as multi-user authentication, cloud storage, or backend database integration. However, it provides a strong foundation for extending the system into a full-stack application in the future.

3.SYSTEM DESIGN

3.1 System Architecture

The Personal Expenses Tracker follows a single-tier (client-side) architecture, where all functionalities are handled within the web browser. The system consists of the Presentation Layer, Application Layer, and Data Layer, but all are implemented on the client side without a backend server. This design ensures simplicity, fast performance, and ease of deployment.

1. Presentation Layer (Frontend)

This layer represents the user interface of the application. It is developed using HTML, CSS, and JavaScript. The frontend allows users to interact with the system by providing features such as adding income and expenses, selecting categories, viewing transaction history, filtering records, and visualizing data through charts. The interface also includes modern UI elements like dark mode, animations, and responsive design for better user experience.

2. Application Layer (Client-side Logic)

This layer handles the business logic of the application using JavaScript. It processes user inputs, validates data, and performs operations such as adding, deleting, and filtering transactions. It also calculates total income, expenses, and balance dynamically. Additionally, it updates the user interface in real time and manages chart visualization using the Chart.js library.

3. Data Layer (Browser Storage)

The data layer is responsible for storing and managing application data using the browser's localStorage. All transaction details, including type, description, amount, category, and date, are stored in JSON format. This ensures data persistence even after refreshing or reopening the application, without requiring a database.

Working Flow:

1. The user interacts with the frontend interface by entering transaction details
2. The application layer (JavaScript) validates the input data
3. The data is stored in localStorage in the browser
4. The application processes the stored data to calculate totals
5. The chart and transaction list are updated dynamically
6. The frontend displays the updated information to the user

3.2 Modules

The Personal Expenses Tracker system is divided into several key modules, each responsible for handling specific functionalities of the application. These modules work together to ensure efficient data handling, real-time updates, and a smooth user experience. The system follows a client-side modular architecture, where each component is designed to perform a well-defined task while interacting seamlessly with other modules.

Module 1: User Input Module

This module handles all user interactions related to entering transaction details. It allows users to input information such as transaction type (income or expense), description, amount, category, and date. The module performs validation checks to ensure that all required fields are filled correctly and that the entered data is valid before processing. By preventing incorrect or incomplete inputs, this module ensures the reliability and accuracy of the data being stored in the system.

Module 2: Transaction Management Module

The transaction management module is responsible for handling all operations related to financial transactions. It enables users to add new transactions and delete existing ones as required. Each transaction is stored as an object containing details such as type, description, amount, category, and date. The module maintains the integrity of the transaction list and ensures that any updates are immediately reflected across the system. It plays a central role in managing the core functionality of the application.

Module 3: Data Storage Module

This module manages the storage and retrieval of transaction data using the browser's Local Storage API. All transaction records are stored in JSON format, allowing structured and efficient data handling. The module ensures that user data persists even after page refresh or browser restart. By eliminating the need for a backend database, this module makes the application lightweight while still providing reliable data persistence.

Module 4: Rendering and Calculation Module

The rendering and calculation module is responsible for dynamically updating the user interface based on the stored data. It calculates total income, total expenses, and the current balance in real time. The module also generates and displays the list of transactions on the screen. It uses DOM manipulation techniques to ensure that any changes in data are instantly reflected in the interface, thereby improving responsiveness and user interaction.

Module 5: Data Visualization Module

This module provides graphical representation of expense data using a pie chart. It utilizes a charting library to display category-wise distribution of expenses, such as Food, Travel, Bills, and Others. The module recalculates category totals whenever a transaction is added or removed and updates the chart accordingly. This visual representation helps users easily analyze their spending patterns and make better financial decisions.

Module 6: Filtering Module

The filtering module allows users to view transactions based on selected categories. By applying filters, users can focus on specific types of expenses without being overwhelmed by the complete dataset. This module enhances usability and provides a more organized way to analyze financial data.

Module 7: Theme Management Module

This module is responsible for implementing the dark mode feature in the application. It allows users to switch between light and dark themes for better visual comfort. The module uses CSS variables and class toggling to dynamically change the appearance of the interface without affecting functionality.

Module 8: User Interface (UI) Module

The UI module handles the overall design and presentation of the application. It is developed using HTML, CSS, and JavaScript, incorporating modern design elements such as gradient backgrounds, glassmorphism effects, animations, and responsive layouts. The module ensures smooth navigation, interactive elements, and an aesthetically pleasing interface, thereby enhancing the overall user experience.

These modules collectively form a modular and event-driven system architecture, ensuring efficient execution, real-time updates, and a clear separation of concerns within the application.

3.3 Backend Design

The current implementation of the Personal Expenses Tracker is a client-side web application that does not include a dedicated backend server. All data processing, storage, and rendering are handled within the browser using JavaScript and Local Storage. However, for scalability, security, and multi-user support, a backend architecture can be designed and integrated into the system.

The proposed backend design follows a client-server architecture, where the frontend interacts with a server through RESTful APIs. The backend can be developed using technologies such as Node.js, Java (Servlets), or Python, and connected to a relational database like MySQL.

1. Server-Side Application Layer

This layer is responsible for handling client requests, processing business logic, and generating appropriate responses. It exposes RESTful API endpoints for operations such as adding transactions, retrieving transaction history, and deleting records. The server validates incoming data and ensures proper handling of requests before interacting with the database.

2. API Design

The backend provides a set of API endpoints to communicate with the frontend. These may include:

- POST /transactions → Add a new transaction
- GET /transactions → Retrieve all transactions
- DELETE /transactions/{id} → Delete a transaction

These APIs follow standard HTTP methods and ensure structured communication between client and server.

3. Database Design

The backend uses a database to store persistent data. A typical table structure for transactions includes:

- Transaction ID (Primary Key)
- Type (Income/Expense)
- Description

- Amount
- Category
- Date

The database ensures data consistency, integrity, and efficient retrieval of records.

4. Authentication and Security Layer

In an extended version, user authentication can be implemented to support multiple users. This includes:

- User registration and login
- Session management or token-based authentication
- Secure access to personal transaction data

This layer ensures that user data is protected and accessible only to authorized users.

5. Data Processing Layer

This layer performs calculations such as total income, total expenses, and balance. While currently handled on the frontend, these operations can be shifted to the backend for better performance and centralized logic in large-scale applications.

6. Integration with Frontend

The frontend communicates with the backend using asynchronous requests (e.g., Fetch API). Data is sent and received in JSON format, ensuring compatibility and ease of processing. The backend responds with updated data, which is then rendered dynamically on the user interface.

Although the current system operates without a backend, the proposed backend design ensures scalability, enhanced security, and support for multi-user environments, making the application suitable for real-world deployment.

3.4 Frontend Design

The frontend of the Personal Expenses Tracker is designed to provide an interactive, responsive, and visually appealing user interface. It is developed using HTML, CSS, and JavaScript, ensuring smooth user interaction and real-time updates without requiring page reloads. The design focuses on usability, clarity, and modern UI principles to enhance the overall user experience.

The application interface is structured using a grid-based layout, dividing the screen into two main sections: the input and transaction management area, and the data visualization area. The left section allows users to enter transaction details, view summaries, and manage transaction records, while the right section displays graphical insights in the form of charts. This structured layout improves readability and efficient navigation within the application.

The input interface consists of form elements such as dropdowns, text fields, number inputs, and date selectors, enabling users to easily enter transaction details like type, description, amount, category, and date. These inputs are designed with proper spacing, rounded corners, and responsive alignment to ensure ease of use. Validation mechanisms are implemented using JavaScript to prevent incorrect or incomplete data entry.

The frontend dynamically updates the user interface using DOM manipulation techniques. Whenever a transaction is added or deleted, the interface instantly reflects the updated list of transactions, along with recalculated values such as total income, total expenses, and current balance. This real-time responsiveness enhances user interaction and eliminates the need for manual refresh.

A key feature of the frontend design is the integration of data visualization, where category-wise expenses are displayed using a pie chart. This graphical representation helps users quickly understand their spending patterns. The chart updates automatically based on the latest transaction data, providing accurate and meaningful insights.

The application also incorporates modern UI/UX design elements to improve visual appeal and engagement. These include animated gradient backgrounds, floating particle effects, glassmorphism-style containers, hover animations, and interactive buttons. Additionally, a dark mode feature is implemented using CSS variables, allowing users to switch between light and dark themes for better accessibility and visual comfort.

The frontend is designed to be responsive, ensuring compatibility across different screen sizes and devices. Flexible layouts and scalable elements allow the application to maintain its structure and usability on various displays.

Overall, the frontend design follows a responsive and event-driven approach, combining dynamic data handling with modern UI techniques to deliver an efficient, user-friendly, and visually engaging application.

4. IMPLEMENTATION

4.1 Module Implementation

The Personal Expenses Tracker is implemented using a modular approach, where each module is developed using JavaScript functions and integrated with the frontend built using HTML and CSS. Each module performs a specific task, ensuring efficient execution and maintainability of the system.

1. User Input Module Implementation

The User Input Module is implemented using HTML form elements such as input fields, dropdown menus, and date selectors. JavaScript retrieves user inputs using DOM methods like `getElementById()`. Input validation is performed to ensure that fields such as description, amount, and date are not left empty and that the amount entered is a valid number. This ensures that only correct and complete data is processed further in the system.

2. Transaction Management Module Implementation

The Transaction Management Module is implemented using JavaScript functions such as `addTransaction()` and `deleteItem()`. When a user adds a transaction, the input data is stored as an object and pushed into an array. Each object contains attributes like type, description, amount, category, and date. The delete functionality removes a transaction from the array using its index, ensuring efficient modification of the dataset.

3. Data Storage Module Implementation

The Data Storage Module is implemented using the browser's Local Storage API. The transaction array is converted into JSON format using `JSON.stringify()` and stored locally. When the application loads, the stored data is retrieved using `JSON.parse()` and assigned back to the array. This ensures that data persists even after refreshing or reopening the application.

4. Rendering and Calculation Module Implementation

The Rendering and Calculation Module is implemented through the `render()` function. This function iterates through the transaction data to calculate total income, total expenses, and the current balance. It also dynamically updates the transaction list by creating and appending HTML elements using DOM manipulation. This ensures real-time updates of the user interface whenever data changes.

5. Data Visualization Module Implementation

The Data Visualization Module is implemented using the `Chart.js` library. The `updateChart()` function processes category-wise expense data and generates a pie chart. It calculates the distribution of expenses across categories and updates the chart dynamically whenever transactions are added or removed, providing users with a clear visual representation of their spending.

6. Filtering Module Implementation

The Filtering Module is implemented using a dropdown menu that allows users to select specific categories. The selected value is used within the `render()` function to filter transactions. Only transactions matching the selected category are displayed, improving data organization and user experience.

7. Theme Management Module Implementation

The Theme Management Module is implemented using a toggle function that switches between light and dark modes. This is achieved by adding or removing a CSS class from the document body, which changes the appearance of elements using predefined CSS variables. This enhances accessibility and visual comfort for users.

8. User Interface (UI) Module Implementation

The UI Module is implemented using HTML for structuring the layout and CSS for styling. Advanced design features such as gradient backgrounds, animations, glassmorphism effects, and responsive grid layouts are incorporated to improve visual appeal. JavaScript is used to enhance interactivity, making the interface dynamic and user-friendly.

4.2 Sample Code

1.index.html:

```
<!DOCTYPE html>

<html lang="en">
<head>
<meta charset="UTF-8">
<title>Personal Expense Tracker</title>
<link rel="stylesheet" href="style.css">
<script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
</head>
<body>
<div class="container">
<h2> Personal Expenses Tracker</h2>
<div class="main">
<!-- LEFT -->
<div>
<select id="type">
<option value="income">Income</option>
<option value="expense">Expense</option>
</select>
<input id="desc" placeholder="Description">
<input id="amount" type="number" placeholder="Amount">
<select id="category">
<option>Food</option>
<option>Travel</option>
<option>Bills</option>
<option>Others</option>
</select>
<input type="date" id="date">
<button onclick="add()">Add</button>
<div class="summary">
<div class="card balance">Balance ₹<span id="bal">0</span></div>
<div class="card income">Income ₹<span id="inc">0</span></div>
<div class="card expense">Expense ₹<span id="exp">0</span></div>
</div>
<ul id="list"></ul>
</div>
<!-- RIGHT -->
<div>
```

```

<h3>Category</h3>
<canvas id="pie"></canvas>
<h3>Monthly</h3>
<canvas id="month"></canvas>
<h3>Trend</h3>
<canvas id="trend"></canvas>
</div>
</div>
</div>
<script src="script.js"></script>
</body>
</html>

```

2. style.css

```

body {
  font-family: Arial;
  margin: 0;
  background: linear-gradient(135deg,#667eea,#764ba2);
}
.container {
  max-width: 1100px;
  margin: 30px auto;
  background: white;
  padding: 20px;
  border-radius: 10px;
}
.main {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 20px;
}
input, select, button {
  padding: 10px;
  margin: 5px;
}
button {
  background: #ff416c;
  color: white;
}

```

```

    border: none;
  }
  .summary {
    display: flex;
    justify-content: space-between;
  }
  .card {
    flex: 1;
    margin: 5px;
    padding: 15px;
    color: white;
    text-align: center;
    border-radius: 8px;
  }
  .balance { background: #3498db; }
  .income { background: #2ecc71; }
  .expense { background: #e74c3c; }
  ul { padding: 0; list-style: none; }
  li {
    background: #eee;
    margin: 5px 0;
    padding: 10px;
  }
  canvas {
    width: 100% !important;
    height: 250px !important;
  }

```

3. script.js

```

alert("JS is running");

let data = [

  {type:"income", desc:"Pocket Money", amt:5000, cat:"Others", date:"2026-04-01"},

  {type:"expense", desc:"Food", amt:200, cat:"Food", date:"2026-04-02"},

  {type:"expense", desc:"Travel", amt:300, cat:"Travel", date:"2026-04-03"}

];

```

```

function add(){
let type = document.getElementById("type").value;
let desc = document.getElementById("desc").value;
let amt = +document.getElementById("amount").value;
let cat = document.getElementById("category").value;
let date = document.getElementById("date").value;
if(!desc || !amt || !date) return alert("fill all");
data.push( {type,desc,amt,cat,date} );
render();
}

function render(){
let list = document.getElementById("list");
list.innerHTML="";
let inc=0,exp=0,bal=0;
let catData={Food:0,Travel:0,Bills:0,Others:0};
let months={},trend=[];
data.forEach(d=>{
let li=document.createElement("li");
li.innerText=d.desc+" ₹"+d.amt;
list.appendChild(li);
let m=new Date(d.date).getMonth();
if(d.type==="income"){
inc+=d.amt; bal+=d.amt;
}else{
exp+=d.amt; bal-=d.amt;

```

```

catData[d.cat]+=d.amt;

}

months[m]=(months[m]||0)+d.amt;

trend.push(bal);

});

document.getElementById("inc").innerText=inc;

document.getElementById("exp").innerText=exp;

document.getElementById("bal").innerText=bal;

draw("pie",Object.keys(catData),Object.values(catData),"pie");

draw("month",Object.keys(months),Object.values(months),"bar");

draw("trend",trend.map((_,i)=>i),trend,"line");

}

function draw(id, labels, data, type) {

const canvas = document.getElementById(id);

if (!canvas) return;

const ctx = canvas.getContext("2d");

if (window[id]) window[id].destroy();

window[id] = new Chart(ctx, {

type: type,

data: {

labels: labels.length ? labels : ["No Data"],

datasets: [{

label: "Amount",

data: data.length ? data : [0],

backgroundColor: [

"#ff6384", "#36a2eb", "#ffcd56",

```

```

"#4bc0c0", "#9966ff", "#ff9f40"
],
borderColor: "#333",
borderWidth: 2
}]
},
options: {
responsive: true,
maintainAspectRatio: false
}
});
}

window.onload = render;

```

4.3 Sample output screenshots

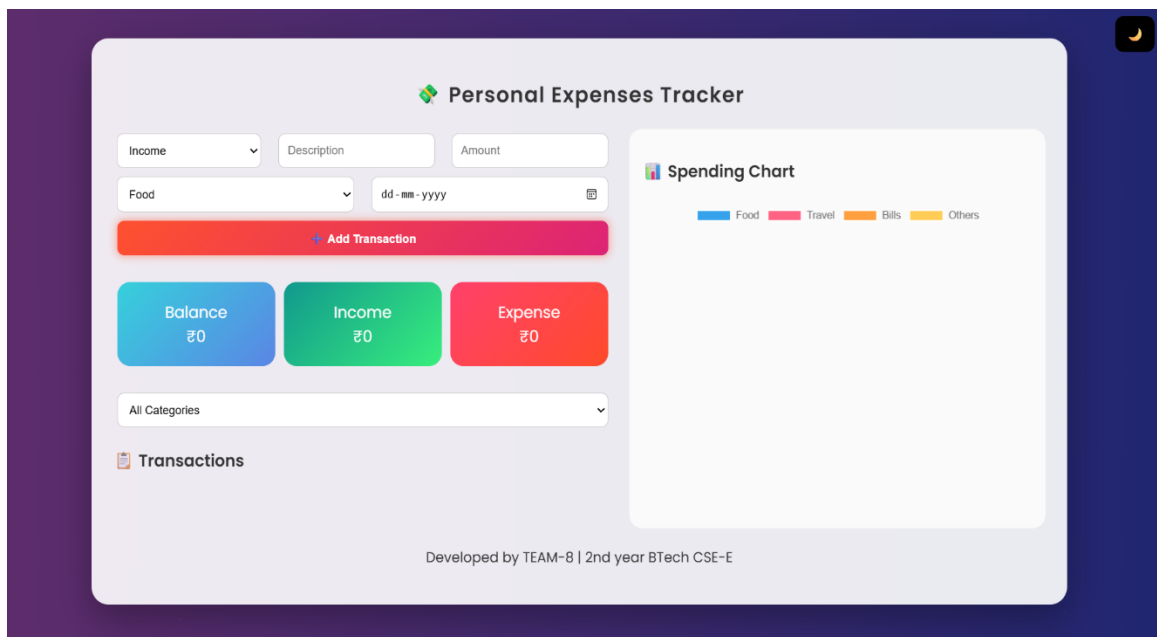


Fig.4.3.1 Main Interface(Home page)

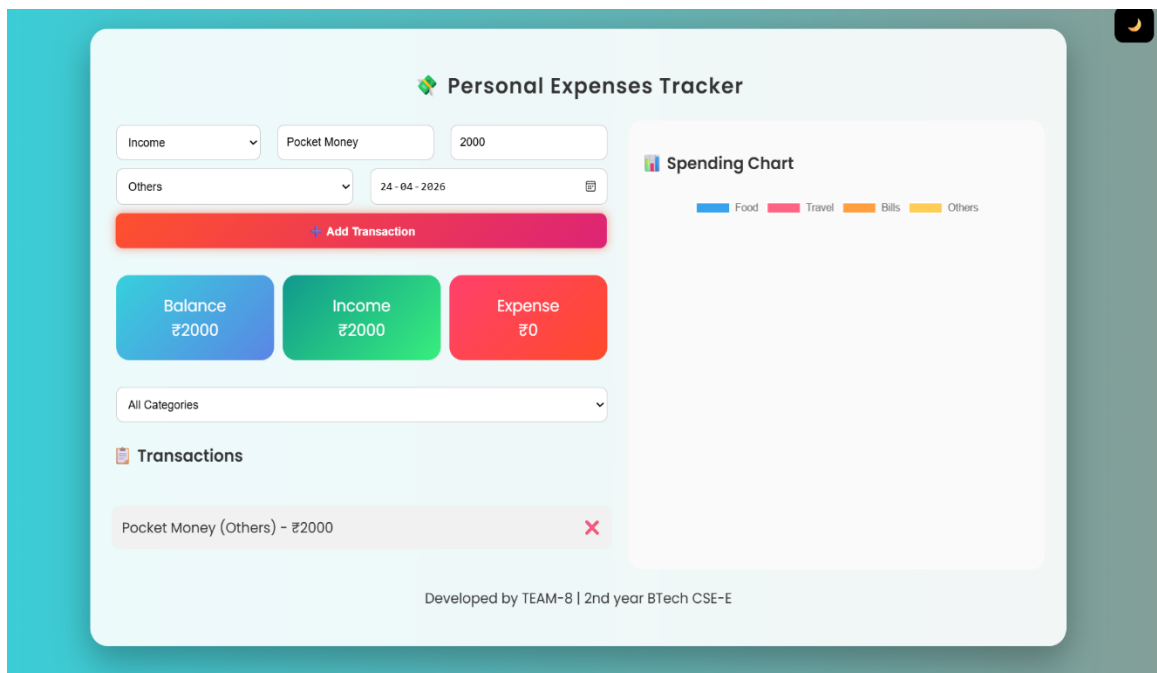


Fig.4.3.2 Adding Transaction

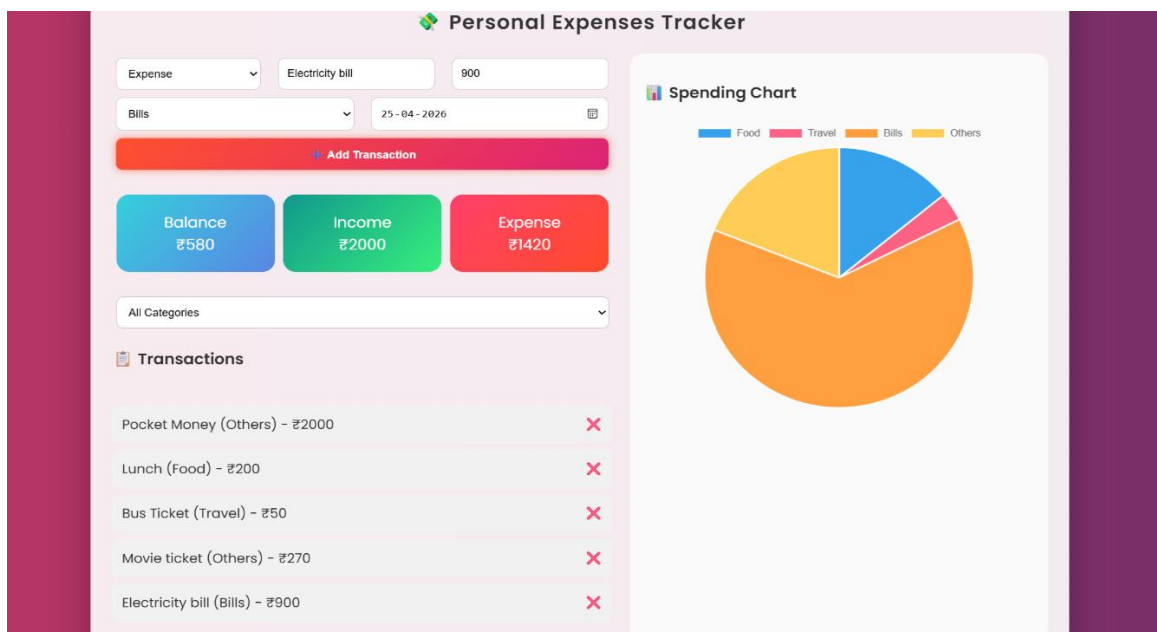


Fig.4.3.3 Transaction List Display

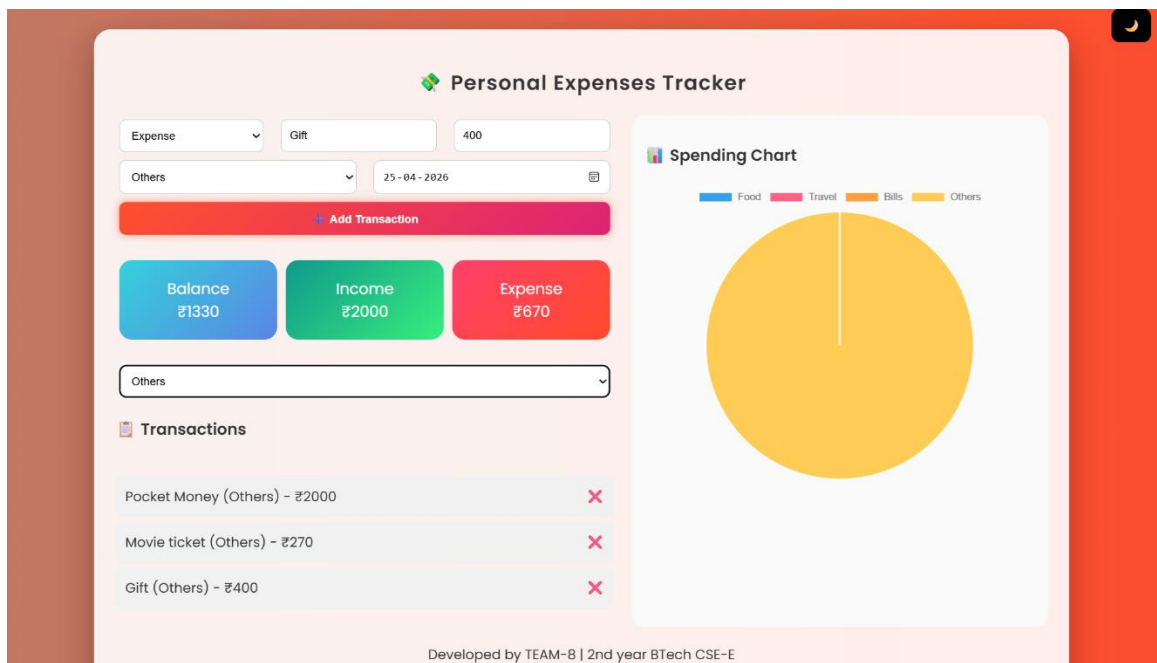


Fig.4.3.4 Category Filtering

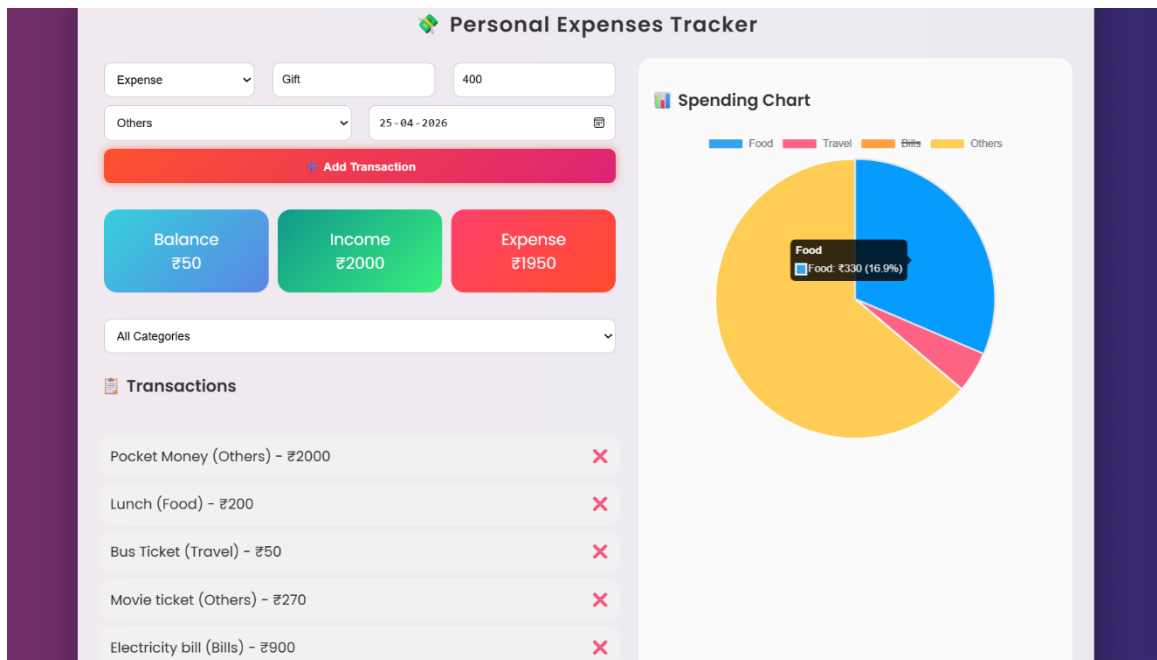


Fig.4.3.5 Expense Visualization Chart

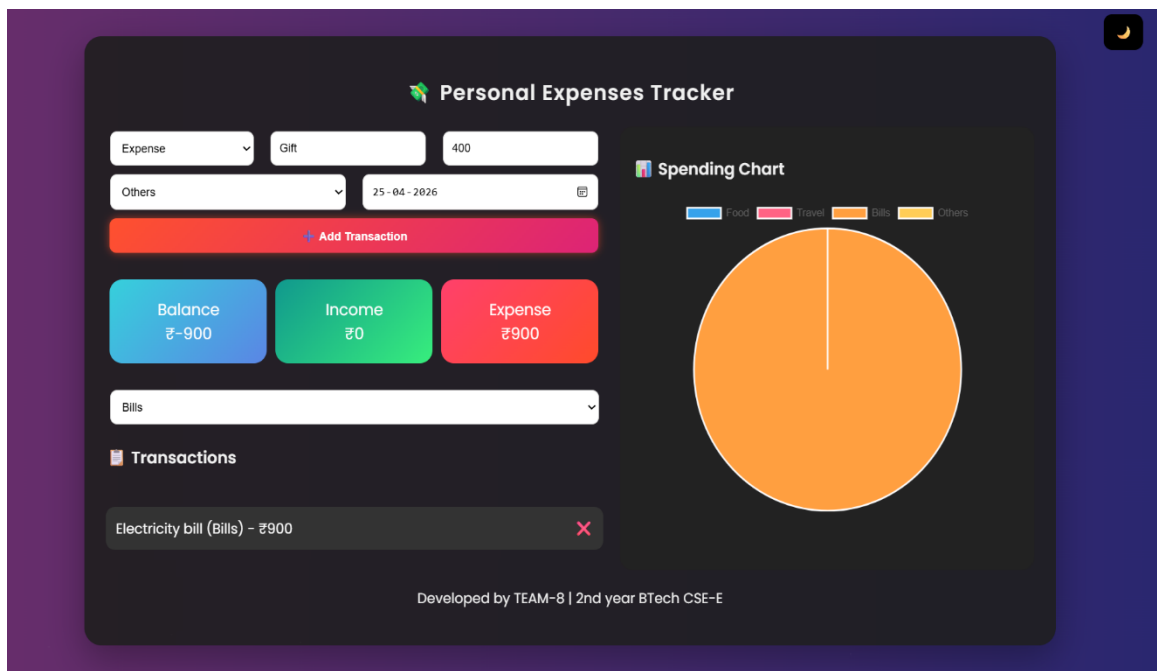


Fig.4.3.6 Dark Mode Interface

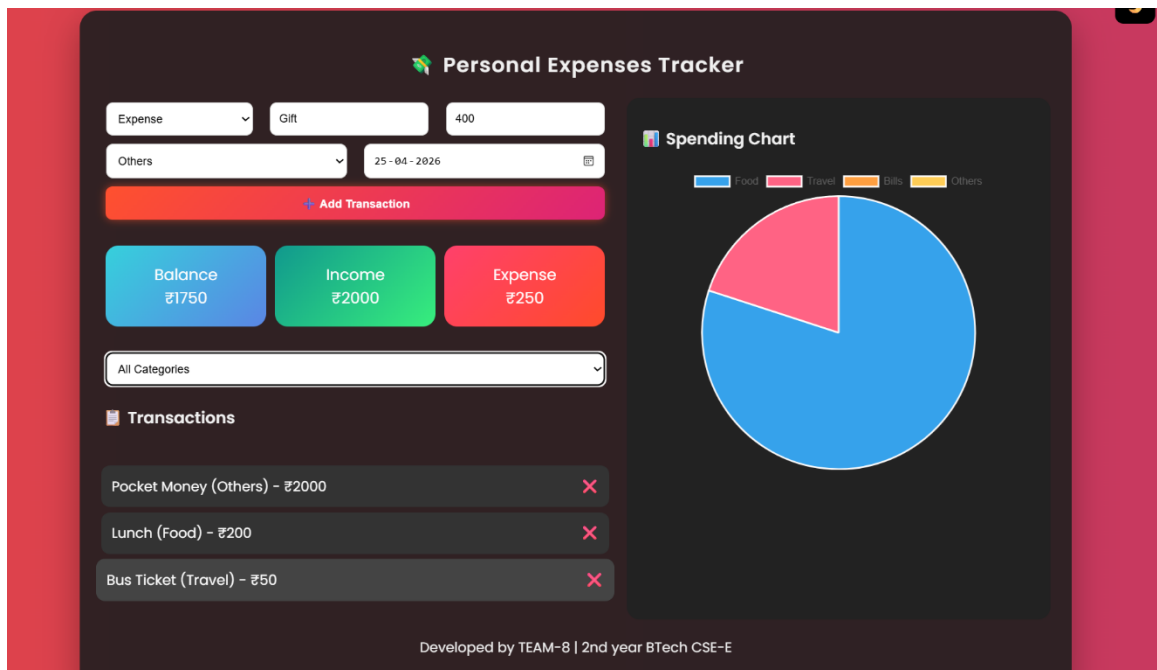


Fig.4.3.7 Delete Transaction Functionality

5. CONCLUSION

The Personal Expenses Tracker web application has been successfully designed and implemented to address the need for efficient and systematic management of personal financial transactions. The system provides a comprehensive platform that enables users to record, categorize, and monitor their income and expenses in a structured manner. By offering real-time updates of financial summaries such as total income, total expenditure, and available balance, the application helps users maintain better control over their financial activities and promotes informed decision-making.

The project effectively demonstrates the application of core web technologies, including HTML for structuring the interface, CSS for styling and responsive design, and JavaScript for implementing dynamic functionality and interactivity. The use of browser-based Local Storage ensures persistent data handling without the need for a backend database, making the system lightweight and efficient. Additionally, the integration of data visualization through graphical representation enhances the analytical capability of the application, allowing users to easily interpret their spending patterns across different categories.

From a design perspective, the system follows a modular and event-driven architecture, ensuring a clear separation of concerns and efficient communication between components. Each module, such as user input handling, transaction management, data storage, rendering, and visualization, is implemented in a structured manner, contributing to the overall maintainability and scalability of the application. The inclusion of modern UI/UX features such as responsive layout, animated backgrounds, and dark mode functionality further improves user engagement and accessibility.

Despite being a client-side application, the project lays a strong foundation for future enhancements. The system can be extended by integrating a backend server and database to support multi-user environments, secure authentication, and cloud-based data storage. Additional features such as detailed financial reports, predictive analysis, and mobile compatibility can further enhance the functionality and usability of the application.

In conclusion, the Personal Expenses Tracker successfully fulfills its intended objectives by providing an intuitive, efficient, and visually interactive platform for financial management. The project not only demonstrates technical proficiency in web development but also highlights the practical relevance of technology in solving everyday problems. It serves as a scalable and adaptable solution that can be further developed into a fully functional real-world financial management system.

6. REFERENCES

- Mozilla Developer Network (MDN), *Web Docs*, Available at: <https://developer.mozilla.org>
(Used for understanding JavaScript, DOM manipulation, and Local Storage API)
- W3Schools, *HTML, CSS, and JavaScript Tutorials*, Available at: <https://www.w3schools.com>
(Used for basic syntax and implementation guidance)
- Chart.js, *Official Documentation*, Available at: <https://www.chartjs.org/docs/latest/>
(Used for implementing pie chart visualization)
- Google Fonts, *Poppins Font Documentation*, Available at: <https://fonts.google.com>
(Used for UI font styling)
- ECMA International, *ECMAScript Language Specification*, Available at: <https://www.ecma-international.org>
(Used for understanding JavaScript standards)
- WHATWG, *HTML Living Standard*, Available at: <https://html.spec.whatwg.org>
(Used for HTML structure and standards)
- CSS Working Group, *CSS Specifications*, Available at: <https://www.w3.org/Style/CSS/>
(Used for styling and responsive design concepts)